



UNIVERSIDADE REGIONAL DO NOROESTE DO
ESTADO DO RIO GRANDE DO SUL
CURSO DE CIÊNCIA DA COMPUTAÇÃO

JEAN KRONBAUER DE MOURA

DESENVOLVIMENTO DE UM *HUB* ADMINISTRATIVO CENTRALIZADO PARA
GESTÃO DE ECOSSISTEMA SAAS: PROJETO OFICINA MASTER

Ijuí/RS

2026

JEAN KRONBAUER DE MOURA

DESENVOLVIMENTO DE UM *HUB* ADMINISTRATIVO CENTRALIZADO PARA
GESTÃO DE ECOSSISTEMA SAAS: PROJETO OFICINA MASTER

Relatório apresentado ao curso de Ciência da
Computação da Universidade Regional do Noro-
este do Estado do Rio Grande do Sul - UNIJUÍ,
como requisito para a obtenção do grau de Ba-
charel em Ciência da Computação.

Orientador(a): Prof(a). Fabricia Car-
neiro Roos Frantz

Ijuí/RS

2026

JEAN KRONBAUER DE MOURA

DESENVOLVIMENTO DE UM *HUB* ADMINISTRATIVO CENTRALIZADO PARA
GESTÃO DE ECOSISTEMA SAAS: PROJETO OFICINA MASTER

Relatório apresentado ao curso de Ciência da
Computação da Universidade Regional do Noro-
este do Estado do Rio Grande do Sul - UNIJUÍ,
como requisito para a obtenção do grau de Ba-
charel em Ciência da Computação, submetido à
aprovação da banca examinadora composta pe-
los seguintes membros:

Prof(a). Fabricia Carneiro Roos Frantz
Orientador(a)

Prof(a). Fabricia Carneiro Roos Frantz
Membro da Banca

Ijuí/RS, XX de Junho de 2026

AGRADECIMENTOS

Agradeço à equipe da FK Oficina pela oportunidade e pelo acolhimento exemplar durante este período de estágio. A confiança depositada em mim para o desenvolvimento de um projeto tão crítico para o ecossistema da empresa foi um motor constante de motivação e aprendizado.

Um agradecimento especial ao meu supervisor técnico, Eduardo Froner, por compartilhar generosamente sua vasta experiência. Suas orientações técnicas e conselhos profissionais foram fundamentais para o sucesso do projeto Oficina Master e para o meu amadurecimento como desenvolvedor.

Agradeço à Universidade Regional do Noroeste do Estado do Rio Grande do Sul (UNIJUÍ) e ao corpo docente do curso de Ciência da Computação, especialmente à minha orientadora Prof(a). Fabricia Carneiro Roos Frantz, cujos ensinamentos e suporte foram essenciais para a fundamentação teórica deste trabalho.

Por fim, agradeço à minha família e amigos, pelo incentivo constante e por compreenderem as horas de dedicação necessárias para a conclusão desta etapa fundamental da minha formação.

RESUMO

Este relatório documenta as atividades realizadas durante o estágio supervisionado focado no desenvolvimento do projeto Oficina Master, um *hub* administrativo centralizado para um ecossistema de gestão de oficinas mecânicas (*SaaS*). O projeto teve como objetivo principal solucionar a complexidade da gestão manual de múltiplos *tenants* e instâncias de banco de dados, além de centralizar o controle de licenciamento. A solução foi desenvolvida utilizando o *framework* *Laravel* 12 no *backend*, integrando-se via *API REST* com um *frontend* moderno construído em *Vue.js* 3 com *Pinia*. Entre as funcionalidades implementadas, destacam-se a gestão automatizada de migrações de banco de dados, propagação de tabelas tributárias (IBPT) e um *dashboard* administrativo para monitoramento do ecossistema. Os resultados obtidos demonstram uma redução significativa no tempo de suporte técnico e na manutenção do ecossistema, consolidando conhecimentos práticos em arquitetura de software, segurança de dados e integração de serviços.

Palavras-chave: *laravel*; *vue.js*; *saas*; arquitetura de software; *api rest*.

LISTA DE ILUSTRAÇÕES

Figura 1 – Visão Geral do Repositório no GitHub e Histórico de Sincronização	17
Figura 2 – Histórico de Contribuições e Versionamento do Projeto	17
Figura 3 – Macro Arquitetura do Ecossistema Oficina Master	22
Figura 4 – Diagrama de Atividades: Provisionamento Automatizado de Nova Instância	24
Figura 5 – Diagrama de Sequência: Alternância Dinâmica de Contexto de Tenant (<i>Tenant Switcher</i>)	26
Figura 6 – <i>Dashboard</i> Administrativo: Painel Integrado de KPIs e Telemetria	29
Figura 7 – Interface de Autenticação do Hub Master	30
Figura 8 – Interfaces de Gestão de Usuários e Controle de Acesso	30
Figura 9 – Listagem Geral de Usuários Cadastrados no Sistema	31
Figura 10 – Listagem Geral de Empresas Cadastradas	31
Figura 11 – Monitoramento de Instâncias de Bancos de Dados	32
Figura 12 – Visualização Consolidada de Planos e Licenças no Ecossistema	32
Figura 13 – Interfaces de Configuração Comercial e Emissão de Licenças	32
Figura 14 – Interface de Alteração e Ajuste de Plano de Empresa	33
Figura 15 – Pipeline de Processamento de Requisição com Injeção de Contexto	37
Figura 17 – Estratégia de Ramificação e Ciclo de Vida do Código	38

LISTA DE ABREVIATURAS E SIGLAS

API	<i>Application Programming Interface</i>
CI/CD	<i>Continuous Integration / Continuous Deployment</i>
DI	<i>Dependency Injection</i> (Injeção de Dependência)
ERP	<i>Enterprise Resource Planning</i>
IBPT	Instituto Brasileiro de Planejamento e Tributação
IoC	<i>Inversion of Control</i> (Inversão de Controle)
JSON	<i>JavaScript Object Notation</i>
JWT	<i>JSON Web Token</i>
LGPD	Lei Geral de Proteção de Dados
MVC	<i>Model-View-Controller</i>
NCM	Nomenclatura Comum do Mercosul
ORM	<i>Object-Relational Mapping</i>
PHP	<i>Hypertext Preprocessor</i>
PSR	<i>PHP Standard Recommendation</i>
RAM	<i>Random Access Memory</i>
RBAC	<i>Role-Based Access Control</i>
REST	<i>Representational State Transfer</i>
SaaS	<i>Software as a Service</i>
SEFAZ	Secretaria da Fazenda
SoC	<i>Separation of Concerns</i>
SPA	<i>Single Page Application</i>
TEF	Transferência Eletrônica de Fundos
UI/UX	<i>User Interface / User Experience</i>
UNIJUÍ	Universidade Regional do Noroeste do Estado do Rio Grande do Sul
WIP	<i>Work In Progress</i>

SUMÁRIO

1	INTRODUÇÃO	9
1.1	JUSTIFICATIVA	10
1.2	OBJETIVOS	10
1.2.1	Objetivo Geral	10
1.3	OBJETIVOS	11
1.3.1	Objetivo Geral	11
1.3.2	Objetivos Específicos	11
1.4	ESTRUTURA DO TRABALHO	11
2	APRESENTAÇÃO DA EMPRESA	12
2.1	HISTÓRICO E EVOLUÇÃO ESTRATÉGICA	12
2.2	MISSÃO, VISÃO E VALORES ORGANIZACIONAIS	12
2.3	ESTRUTURA E ECOSSISTEMA TECNOLÓGICO	13
2.3.1	Arquitetura de Produto e Filosofia <i>Multi-tenancy</i>	13
2.3.2	Stack Tecnológica e Ferramental de Desenvolvimento	13
3	METODOLOGIA	15
3.1	DEFINIÇÃO DE ESCOPO E VISÃO DE PRODUTO	15
3.2	FLUXO DE DESENVOLVIMENTO ORIENTADO A <i>FEATURES</i>	15
3.3	CONTROLE DE VERSÃO COM <i>GIT</i>	16
3.4	VALIDAÇÃO E ALINHAMENTO CONTÍNUO	16
4	REVISÃO BIBLIOGRÁFICA	18
4.1	ARQUITETURA SAAS E PARADIGMAS DE <i>MULTI-TENANCY</i>	18
4.2	O ECOSSISTEMA <i>LARAVEL: SERVICE CONTAINER</i> E INJEÇÃO DE DEPENDÊNCIAS	18
4.2.1	<i>Service Layer Pattern</i> e os Princípios de <i>Clean Architecture</i>	19
4.3	<i>FRONTEND</i> MODERNO: REATIVIDADE, <i>SPA</i> E GESTÃO DE ESTADO .	19
4.3.1	<i>Pinia</i>: O Novo Padrão de Gestão de Estado Reativo	19
4.4	SEGURANÇA EM <i>APIS REST</i> E PROTOCOLOS DE AUTENTICAÇÃO .	19
5	IMPLEMENTAÇÃO E ORQUESTRAÇÃO DO ECOSSISTEMA OFI- CINA MASTER	21
5.1	MACRO ARQUITETURA E ESTRATÉGIA DE ISOLAMENTO DE DADOS	21
5.2	ORGANIZAÇÃO DO PROJETO E PADRÕES DE IMPLEMENTAÇÃO . .	22
5.2.1	Estrutura de Pastas e Namespaces	22
5.2.2	Camada de Modelagem: Eloquent e Relacionamentos de Domínio	23

5.3	PROVISIONAMENTO AUTOMATIZADO E ORQUESTRAÇÃO DE INSTÂNCIAS	23
5.3.1	TenantDatabaseService: Motor de Provisionamento e Estratégia de Inicialização	24
5.3.2	Mecanismo de Rollback e Garantias de Consistência	25
5.4	GESTÃO CENTRALIZADA E HUB DE BANCOS DE DADOS	25
5.4.1	Tenant Switching: Alternância Dinâmica de Contexto de Conexão	25
5.4.2	Telemetria e Observabilidade das Instâncias	26
5.5	ADMINISTRAÇÃO DE PLANOS E LICENCIAMENTO FLEXÍVEL	27
5.5.1	Estrutura de Planos e Configuração de Módulos	27
5.5.2	Ciclo de Vida do Licenciamento e Controle de Acesso	28
5.6	DASHBOARD ADMINISTRATIVO E INTERFACE REATIVA	28
5.6.1	KPIs e Métricas Agregadas por Domínio	28
5.6.2	Arquitetura Reativa da Interface e Estratégia de Atualização	29
5.7	DEMONSTRAÇÃO DAS INTERFACES DE GESTÃO E EVIDÊNCIAS	29
5.7.1	Acesso e Gestão de Usuários	30
5.7.2	Gestão de Tenants, Planos e Licenciamento	30
5.7.3	Evidências de Desenvolvimento e Controle de Versão	30
6	CONSIDERAÇÕES FINAIS	34
6.1	ANÁLISE CRÍTICA DAS DECISÕES ARQUITETURAIS	34
6.2	AVALIAÇÃO DO IMPACTO NO ECOSSISTEMA FK OFICINA	34
6.3	REFLEXÃO SOBRE O CRESCIMENTO PROFISSIONAL	35
	REFERÊNCIAS	36
	APÊNDICE A – DIAGRAMAS E FLUXOS TÉCNICOS AVANÇADOS	37
A.1	PIPELINE DE REQUISIÇÃO E MIDDLEWARE DE CONTEXTO	37
A.2	ARQUITETURA DE ESTADO NO FRONTEND (PINIA STORES)	37
A.3	ESTRUTURA DE DIRETÓRIOS DO PROJETO (BACKEND)	37
A.4	FLUXO DE TRABALHO GITFLOW	38
	ANEXO A – DOCUMENTAÇÃO DE REFERÊNCIAS TÉCNICAS	39
A.1	DICIONÁRIO DE DADOS DO HUB MASTER	39
A.1.1	Entidade: Tenant (Configurações de Instância)	39
A.2	ESPECIFICAÇÃO COMPLETA DE ENDPOINTS (API MASTER)	39
A.2.1	Autenticação e Segurança	39
A.2.2	Gestão de Tenants e Infraestrutura	39
A.3	PADRÃO DE RESPOSTA JSON (STANDARD API RESPONSE)	40
A.4	CONSULTAS DE TELEMETRIA SQL (REFERÊNCIA)	40

1 INTRODUÇÃO

A ascensão do modelo de entrega de software conhecido como *Software as a Service* (*SaaS*) transformou a forma como empresas de pequeno e médio porte consomem tecnologia. No cenário atual, a digitalização deixou de ser apenas uma ferramenta de suporte para se tornar parte central da prestação de serviços técnicos. Em setores que variam desde oficinas mecânicas até laboratórios especializados em microeletrônica, a capacidade de gerenciar dados de forma precisa e segura é um fator relevante para a sustentabilidade operacional.

No Brasil, esse setor enfrenta desafios específicos, como a volatilidade das regras tributárias e a necessidade de disponibilidade em ambientes muitas vezes sem infraestrutura de conectividade estável. A FK Oficina, inserida neste contexto, atua como provedora de soluções digitais integradas voltadas a mitigar essas complexidades. O ecossistema oferecido pela empresa busca resolver problemas comuns nesses setores, como a falta de rastreabilidade em ordens de serviço, a ineficiência no controle de estoques de peças e a dificuldade de manter a conformidade com o sistema tributário nacional.

Contudo, o crescimento da base de usuários trouxe desafios de infraestrutura relevantes. À medida que o número de *tenants* (instâncias isoladas de clientes) aumentou, a manutenção manual ou semi-automatizada tornou-se um risco operacional. Cada novo cliente representa não apenas uma nova base de dados isolada, necessária para garantir a segurança e a privacidade das informações, mas também um novo ponto de manutenção que exige atualizações de esquema, sincronização de dados tributários e monitoramento de performance. O esforço para manter essa infraestrutura de forma individualizada passou a crescer proporcionalmente ao número de clientes, criando um gargalo técnico que limitava a agilidade no desenvolvimento de novas funcionalidades.

Surge, então, a necessidade do projeto **Oficina Master**. Concebido como um *hub* administrativo centralizado, o Oficina Master atua como a camada de orquestração de todo o ecossistema *SaaS* da FK Oficina. Ele não é apenas uma interface de visualização, mas um sistema de automação projetado para gerenciar o ciclo de vida de cada *tenant*, desde o provisionamento inicial de recursos até a gestão de licenciamento e conformidade fiscal. Este projeto representa a transição da organização para um modelo de gestão de infraestrutura mais estruturado, reduzindo a carga operacional da equipe técnica.

A implementação deste *hub* centralizado permite uma visão unificada sobre a saúde técnica e financeira da rede de clientes. Aspectos como a integridade das transações de pagamento, a validade das licenças de uso e a precisão das alíquotas de impostos (via integração IBPT) passam a ser tratados de forma automatizada. Isso permite que a FK Oficina continue sua expansão sem comprometer a qualidade do serviço ou a segurança dos dados de seus clientes.

1.1 JUSTIFICATIVA

A relevância deste trabalho fundamenta-se na necessidade de escalabilidade, segurança e automação em ambientes de software modernos. Antes da concepção do Oficina Master, rotinas como a migração de esquemas de banco de dados e a atualização das tabelas de NCM (Nomenclatura Comum do Mercosul) eram processos fragmentados e suscetíveis a falhas. A justificativa técnica reside na necessidade de eliminar esses gargalos, permitindo que alterações estruturais que antes demandavam intervenção manual sejam executadas de forma centralizada, com rastreabilidade e mecanismos de *rollback*.

Do ponto de vista estratégico, a justificativa vincula-se à viabilidade do modelo de negócio *SaaS* da FK Oficina. A automação do ciclo de licenciamento permite que a gestão dos clientes ocorra de forma autônoma. Isso reduz a necessidade de intervenção humana em processos administrativos e contribui para um fluxo de trabalho mais previsível e para a redução de falhas no processo de renovação.

Sob a ótica acadêmica e profissional, o desenvolvimento deste projeto proporcionou contato com padrões de projeto e arquiteturas de sistemas distribuídos. O desafio de implementar um mecanismo de alternância dinâmica de contexto de banco de dados (*Tenant Switcher*) no *framework Laravel 12* permitiu a aplicação prática de conceitos como inversão de controle (*IoC*), injeção de dependências (*DI*) e segurança em nível de aplicação. Este trabalho serve como estudo de caso sobre como fundamentos da Ciência da Computação podem ser aplicados a problemas reais de concorrência de dados, segurança da informação em conformidade com a LGPD e disponibilidade em ambientes de nuvem.

1.2 OBJETIVOS

1.2.1 Objetivo Geral

Este relatório documenta as atividades realizadas durante o estágio supervisionado focado no desenvolvimento do projeto Oficina Master, um *hub* administrativo centralizado para um ecossistema de gestão de oficinas mecânicas (*SaaS*). O projeto teve como objetivo principal solucionar a complexidade da gestão manual de múltiplos *tenants* e instâncias de banco de dados, além de centralizar o controle de licenciamento. A solução foi desenvolvida utilizando o *framework Laravel 12* no *backend*, integrando-se via *API REST* com um *frontend* moderno construído em *Vue.js 3* com *Pinia*. Entre as funcionalidades implementadas, destacam-se a gestão automatizada de migrações de banco de dados, o provisionamento de novas instâncias e um *dashboard* administrativo para monitoramento do ecossistema.

1.3 OBJETIVOS

1.3.1 Objetivo Geral

Projetar, desenvolver e implantar o *Oficina Master*, um *hub* administrativo centralizado para a orquestração de um ecossistema SaaS multi-tenant, com foco na automação da gestão de instâncias e no monitoramento da integridade do ecossistema da FK Oficina.

1.3.2 Objetivos Específicos

Para atingir a meta proposta, o projeto foi subdividido nos seguintes objetivos específicos:

- a) Desenvolver uma API RESTful utilizando o *framework* Laravel 12, adotando tipagem estrita, arquitetura em camadas de serviço (*Service Layer*) e princípios SOLID;
- b) Construir uma interface administrativa com Vue.js 3, empregando a *Composition API* e Pinia para gerenciamento de estado global;
- c) Implementar um mecanismo de *Tenant Switching* que permita a alternância dinâmica entre bancos de dados isolados em tempo de execução, viabilizando operações de manutenção e consultas sem impacto aos demais clientes;
- d) Desenvolver rotinas para o provisionamento automatizado de novas empresas, incluindo a criação de banco de dados e injeção de configurações iniciais;
- e) Estabelecer um painel de monitoramento consolidado para acompanhamento de logs de acesso administrativo e métricas de saúde dos serviços.

1.4 ESTRUTURA DO TRABALHO

Este relatório está organizado para guiar o leitor pelas principais fases do projeto. O Capítulo 2 contextualiza a FK Oficina, sua trajetória e posicionamento no mercado. O Capítulo 3 descreve as práticas de desenvolvimento, ferramentas de controle de versão e o ambiente técnico empregado. O Capítulo 4 apresenta a fundamentação teórica sobre arquitetura SaaS, padrões de projeto e as tecnologias utilizadas. O Capítulo 5 expõe a implementação técnica, os desafios encontrados e as soluções adotadas. Por fim, o Capítulo 6 sintetiza as considerações finais, avalia os resultados frente aos objetivos e aponta possíveis evoluções do sistema.

2 APRESENTAÇÃO DA EMPRESA

A FK Oficina é uma organização de tecnologia de vanguarda, especializada no desenvolvimento e na sustentação de soluções de gestão empresarial sob o modelo *Software as a Service (SaaS)*. Com sede estratégica em Augusto Pestana, no Noroeste do Rio Grande do Sul, a empresa atua como um parceiro tecnológico essencial para prestadores de serviços técnicos em todo o território nacional. O foco primordial da organização reside nos setores automotivo e de assistência técnica em eletrônicos, onde a empresa destaca-se pela capacidade singular de traduzir necessidades operacionais densas em fluxos de trabalho simplificados e automatizados.

2.1 HISTÓRICO E EVOLUÇÃO ESTRATÉGICA

A trajetória da FK Oficina é um exemplo notável de inovação pragmática impulsionada pela vivência real do mercado. A organização originou-se no segmento de reparação de eletrônicos, operando inicialmente como uma assistência técnica convencional. Durante esse período, a equipe fundadora enfrentou diretamente os desafios sistêmicos do setor: o controle impreciso de ordens de serviço, a dificuldade na gestão de componentes sensíveis e a complexidade na comunicação com o cliente final. Diante da inexistência de softwares no mercado que atendessem satisfatoriamente a essas dores específicas, a equipe decidiu desenvolver sua própria solução de gestão.

O primeiro sistema concebido serviu como um "laboratório vivo", onde cada funcionalidade era exaustivamente testada sob o rigor do cotidiano operacional da própria empresa. Com a validação interna do modelo e a percepção de que outros empresários do setor enfrentavam obstáculos idênticos, a FK Oficina iniciou um processo de expansão estratégica. A solução evoluiu rapidamente de um software de uso local para um ecossistema complexo em nuvem, incorporando módulos de *ERP (Enterprise Resource Planning)*, emissão de documentos fiscais eletrônicos em conformidade com as diversas SEFAZ estaduais e municipais, e integração nativa com múltiplos meios de pagamento e TEF. Atualmente, a FK Oficina não apenas fornece ferramentas de software, mas sustenta uma infraestrutura tecnológica robusta que apoia o crescimento e a profissionalização de centenas de empresas em todo o Brasil.

2.2 MISSÃO, VISÃO E VALORES ORGANIZACIONAIS

A identidade organizacional da FK Oficina é pautada por um compromisso inegociável com a excelência técnica e o sucesso sustentável de seus clientes:

- **Missão:** Empoderar prestadores de serviços técnicos através de ferramentas de gestão inteligentes e resilientes, removendo a complexidade burocrática e permitindo que o foco

dos nossos parceiros permaneça na qualidade técnica da entrega do serviço.

- **Visão:** Consolidar-se como a principal referência nacional e líder de mercado em ecossistemas *SaaS* para oficinas e assistências técnicas, sendo reconhecida pela inovação constante, pela estabilidade de seus sistemas e pela transparência radical em sua infraestrutura.
- **Valores:** Pragmatismo tecnológico, segurança absoluta de dados, transparência no relacionamento com o ecossistema, cultura de aprendizado contínuo e uma busca incessante pela simplicidade na resolução de problemas complexos.

2.3 ESTRUTURA E ECOSSISTEMA TECNOLÓGICO

A estrutura da FK Oficina é desenhada para suportar cenários de alta disponibilidade, escalabilidade horizontal e segurança de nível corporativo. Sob a liderança técnica de Eduardo Froner, a equipe de desenvolvimento opera sob uma filosofia de modernização constante e dívida técnica controlada, garantindo que o ecossistema acompanhe as rápidas evoluções do mercado financeiro e as frequentes mudanças regulatórias tributárias do país.

2.3.1 Arquitetura de Produto e Filosofia *Multi-tenancy*

O diferencial competitivo e a base da estabilidade da FK Oficina residem em sua arquitetura *multi-tenant* baseada em isolamento de banco de dados (*database-per-tenant*). Diferente de soluções concorrentes que compartilham uma única base de dados lógica entre todos os clientes — o que pode levar a vazamento de dados e degradação mútua de performance — a FK Oficina garante que cada cliente possua sua própria instância lógica e física de dados. Esta abordagem proporciona:

- **Segurança de Dados:** Isolamento total de informações sensíveis, em conformidade com as melhores práticas de privacidade e segurança da informação;
- **Performance Previsível:** Recursos computacionais e de armazenamento que podem ser escalonados de forma independente por cliente;
- **Flexibilidade Operacional:** Capacidade de realizar manutenções, *backups* e migrações de forma granular, sem impactar a disponibilidade da rede global.

2.3.2 Stack Tecnológica e Ferramental de Desenvolvimento

Para sustentar e evoluir essa arquitetura complexa, a empresa emprega uma combinação criteriosamente selecionada de tecnologias de ponta, as quais foram o objeto central de estudo e aplicação durante este estágio:

- **Backend:** PHP 8.2+ utilizando o *framework Laravel 12*, com foco absoluto em tipagem estrita, injeção de dependências e arquitetura baseada em domínios;

- **Frontend:** *Vue.js 3* integrando *Pinia* para gestão de estado reativo e *Vite* para fluxos de build ultra-rápidos, resultando em *Single Page Applications (SPA)* de alta performance;
- **Infraestrutura de Dados:** Utilização estratégica de *PostgreSQL* e *MySQL*, gerenciados por uma camada de abstração dinâmica capaz de lidar com a orquestração de centenas de conexões simultâneas;
- **Integrações Estratégicas:** Consumo de sistemas de telemetria para monitoramento proativo e serviços auxiliares de suporte ao ecossistema.

A FK Oficina posiciona-se, portanto, não apenas como uma *software house*, mas como um motor de transformação digital e eficiência operacional para o setor de serviços, onde a tecnologia é o meio fundamental para alcançar a excelência no atendimento ao cliente final.

3 METODOLOGIA

O desenvolvimento do projeto Oficina Master adotou uma abordagem metodológica pragmática e adaptada à realidade de um projeto conduzido por um único desenvolvedor. Em vez de replicar frameworks projetados para equipes — como Scrum ou SAFe —, optou-se por um processo autogerenciado, orientado à entrega de funcionalidades completas e verticalmente integradas, fundamentado no uso do sistema de controle de versão *Git* e guiado por uma visão de produto definida pelo gestor responsável.

3.1 DEFINIÇÃO DE ESCOPO E VISÃO DE PRODUTO

O ponto de partida do processo metodológico foi uma etapa de alinhamento estratégico entre o gestor do projeto e o desenvolvedor. Nessa fase, o gestor apresentou a visão geral do sistema — seus objetivos de negócio, os módulos essenciais e as prioridades de alto nível —, sem a necessidade de especificações formais exaustivas ou documentação de requisitos completa de antemão.

Essa transferência de visão ocorreu por meio de conversas diretas e objetivas, nas quais o desenvolvedor absorveu o contexto do problema, fez perguntas pontuais de esclarecimento e, a partir disso, conduziu de forma autônoma o detalhamento técnico de cada funcionalidade. Conforme destacado por Pressman (2011), em projetos de menor escala e com equipes reduzidas, a comunicação direta e a autonomia técnica do desenvolvedor tendem a substituir com vantagem os rituais formais de cerimônias ágeis.

3.2 FLUXO DE DESENVOLVIMENTO ORIENTADO A *FEATURES*

Em substituição ao modelo de entrega baseado em ciclos fixos de tempo (*Sprints*), adotou-se um modelo orientado a funcionalidades completas. Cada unidade de trabalho correspondia a uma *feature* funcional do sistema — um módulo ou comportamento coeso e testável de ponta a ponta — que só era considerado concluído quando estivesse integralmente implementado, do banco de dados à interface do usuário.

Esse modelo apresentou vantagens práticas relevantes para o contexto do projeto:

- Eliminação da pressão artificial de prazos semanais, permitindo que funcionalidades complexas — como o mecanismo de *Tenant Switching* e a camada de autenticação de *APIs* — recebessem o tempo necessário para uma implementação robusta;
- Entregas sempre em estado funcional e utilizável, facilitando a validação incremental pelo gestor ao final de cada módulo concluído;

- Maior clareza sobre o progresso real do projeto, uma vez que o critério de conclusão era objetivo: a funcionalidade funciona ou não funciona.

O sequenciamento das *features* foi definido com base em suas dependências técnicas e na prioridade de negócio indicada pelo gestor. Módulos centrais da arquitetura foram desenvolvidos primeiro, estabelecendo uma base sólida sobre a qual as funcionalidades periféricas puderam ser construídas com segurança.

3.3 CONTROLE DE VERSÃO COM *GIT*

O *Git* foi utilizado como ferramenta central de controle de versão e rastreabilidade do trabalho realizado. A estratégia de ramificação adotada foi simples e adequada ao contexto de desenvolvedor solo: para cada nova funcionalidade, criava-se uma *branch* dedicada a partir do ramo principal, onde todo o desenvolvimento era realizado de forma isolada. Ao término e validação local da *feature*, o código era integrado ao ramo principal por meio de um *merge* direto.

Essa abordagem garantiu:

- Histórico de desenvolvimento limpo e organizado por funcionalidade, facilitando eventuais reversões ou análises futuras;
- Isolamento seguro entre funcionalidades em desenvolvimento simultâneo, mesmo que conduzidas pelo mesmo desenvolvedor em momentos distintos;
- Rastreabilidade direta entre commits e as funcionalidades às quais se referem, por meio de mensagens de commit descritivas e padronizadas.

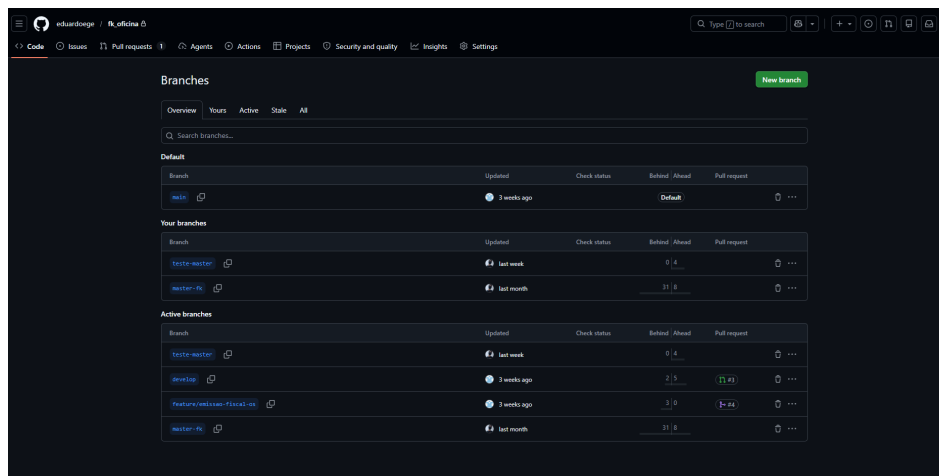
O histórico de contribuições documenta a evolução incremental da base de código, conforme demonstrado na Figura 1 e na Figura 2, refletindo o rigor aplicado no versionamento do projeto.

Não foram adotadas práticas de *Code Review* por pares nem esteiras formais de integração contínua, uma vez que a natureza individual do projeto tornava tais práticas desnecessárias. A qualidade do código foi assegurada pela disciplina do próprio desenvolvedor, pelo uso de ferramentas de análise estática e pela validação funcional a cada entrega.

3.4 VALIDAÇÃO E ALINHAMENTO CONTÍNUO

Ao final do desenvolvimento de cada *feature*, realizava-se uma breve sessão de validação com o gestor. Nesse momento, a funcionalidade era demonstrada em ambiente local ou de homologação, coletavam-se eventuais ajustes ou redirecionamentos e definia-se informalmente a próxima funcionalidade a ser desenvolvida.

Figura 1 – Visão Geral do Repositório no GitHub e Histórico de Sincronização



Fonte: Própria do autor (2026).

Figura 2 – Histórico de Contribuições e Versionamento do Projeto

```

1 * b87b445 gerenciamento dos usuarios do master
2 * a67b8d6 update licenca e plano
3 * 374cdb0 UPDATE INTEGRACAO MASTER COM OFICINA
4 * 567aeb4 teste master
5 * dd7abfa Release: Modulo Fiscal completo + clonagem NF-e
6 * 7092d15 Merge pull request #2 from eduardoage/develop
7 * 615c231 Merge branch 'develop' of fk_oficina
8 * 66ed4bc Merge branch 'feature/emissao-fiscal-os'
9 * ff3ffb3 fix(fiscal): exhibe PDF e bloqueia transmissao
10 * 98b998f feat(fiscal): clonar NF-e e suporte a origem clone
11 * f8d4761 feat(master): registra logs de login

```

Fonte: Própria do autor (2026).

Esse ciclo curto de *feedback* — sem a burocracia de cerimônias formais — permitiu manter o alinhamento entre a visão de produto do gestor e as decisões técnicas do desenvolvedor ao longo de todo o projeto, garantindo que cada entrega agregasse valor real e mensurável ao sistema.

Esta metodologia, embora desprovida de cerimônias formais, demonstrou-se eficaz e adequada ao perfil do projeto: ágil na comunicação, disciplinada no versionamento e focada na entrega de valor funcional a cada iteração.

4 REVISÃO BIBLIOGRÁFICA

Este capítulo apresenta uma fundamentação teórica densa sobre as arquiteturas, padrões de projeto e tecnologias de vanguarda que serviram de alicerce para a concepção e o desenvolvimento do Oficina Master. A compreensão profunda desses conceitos é imperativa para justificar as escolhas arquiteturais do sistema, com foco inabalável em escalabilidade, isolamento rigoroso de dados e reatividade de interface. Conforme postulado por Pressman (2011), uma base teórica sólida e bem articulada é o elemento que distingue o desenvolvimento assistemático da verdadeira engenharia de software profissional e resiliente.

4.1 ARQUITETURA SAAS E PARADIGMAS DE *MULTI-TENANCY*

O modelo *Software as a Service (SaaS)* representa uma mudança de paradigma fundamental na entrega de software corporativo, onde a aplicação e seus dados associados são gerenciados e hospedados centralmente. De acordo com Bezemer e Zaidman (2010), a espinha dorsal técnica do *SaaS* é o *multi-tenancy*, uma abordagem arquitetural que permite que uma única instância lógica da aplicação atenda de forma segura a múltiplos clientes (*tenants*).

No âmbito do projeto Oficina Master, adotou-se o padrão de excelência de **Banco de Dados Isolado** (*database-per-tenant*). Esta abordagem, embora imponha uma complexidade superior na orquestração de infraestrutura e na execução de migrações de esquema, oferece o mais alto nível de segurança, privacidade e isolamento de falhas. Segundo Krebs, Momm e Kounev (2012), esse padrão é superior ao compartilhamento de tabelas, pois facilita processos granulares de *backup* e *restore* por cliente, além de mitigar definitivamente o problema do "vizinho barulhento" (*noisy neighbor*), onde o pico de carga de um locatário poderia comprometer a performance dos demais. Esta estratégia é, ademais, um pilar fundamental para a conformidade técnica com a Lei Geral de Proteção de Dados (LGPD), garantindo o isolamento físico-lógico das bases de dados.

4.2 O ECOSISTEMA LARAVEL: *SERVICE CONTAINER* E INJEÇÃO DE DEPENDÊNCIAS

O *Laravel 12* foi selecionado como o motor de *backend* devido à sua sofisticação arquitetural e maturidade de ecossistema. Para além do tradicional padrão *MVC*, o *framework* destaca-se pela implementação de um robusto e flexível *Service Container*. Segundo Otwell (2024), a Injeção de Dependência (*DI*) é um padrão de projeto estrutural que permite a inversão de controle (*IoC*), promovendo o desacoplamento entre as classes e elevando substancialmente a testabilidade e a flexibilidade do código-fonte.

No desenvolvimento do Oficina Master, a utilização estratégica de *Service Providers* foi vital para orquestrar dinamicamente as conexões com os bancos de dados dos *tenants*. Esta

camada de abstração permite que a aplicação alterne contextos de conexão de forma transparente para a camada de lógica de negócio, seguindo os princípios de *Separation of Concerns (SoC)*.

4.2.1 *Service Layer Pattern* e os Princípios de *Clean Architecture*

Para mitigar a complexidade em um sistema que exige múltiplas integrações e regras de negócio densas, adotou-se o *Service Layer Pattern*. Conforme definido por Fowler (2002), a Camada de Serviço estabelece uma fronteira clara para a aplicação, encapsulando a orquestração de lógica de negócio e regras de domínio em classes de serviço dedicadas e reutilizáveis.

Esta prática permitiu que os controladores da *API* permanecessem concisos e focados estritamente na gestão de entradas e saídas HTTP (*Lean Controllers*), delegando a complexidade de regras de negócio e orquestração de contexto a serviços especializados. Esta abordagem converge diretamente com os princípios de *Clean Architecture* defendidos por Martin (2017), onde as regras de negócio centrais são mantidas isoladas de detalhes externos, como drivers de banco de dados ou interfaces de *API*.

4.3 *FRONTEND* MODERNO: REATIVIDADE, *SPA* E GESTÃO DE ESTADO

A modernização da interface administrativa fundamentou-se no uso do **Vue.js 3**, operando sobre a **Composition API**. Esta abordagem permite uma organização de lógica muito mais modular, tipada e reutilizável quando comparada à tradicional *Options API*. A reatividade do Vue, baseada no uso de *Proxies* nativos do JavaScript moderno, garante que o estado da interface reflita as mutações de dados em tempo real, sem a necessidade de manipulação imperativa do DOM.

4.3.1 *Pinia*: O Novo Padrão de Gestão de Estado Reativo

Para o gerenciamento de estado global e persistência de dados entre rotas, empregou-se a biblioteca **Pinia**. Segundo You (2024), o *Pinia* oferece um sistema de tipos robusto e uma arquitetura baseada em *stores* modulares que simplificam o fluxo de dados em *Single Page Applications (SPA)* de grande escala. No Oficina Master, o *Pinia* foi essencial para sincronizar informações críticas, como o contexto do *tenant* selecionado e o status de autenticação do usuário Master, de forma previsível e performática.

4.4 SEGURANÇA EM *APIS REST* E PROTOCOLOS DE AUTENTICAÇÃO

A exposição de recursos de gestão centralizada exige protocolos de segurança de nível bancário. O Oficina Master adota integralmente os princípios da arquitetura **REST** (*Representational State Transfer*), utilizando métodos HTTP sem estado (*stateless*). A autenticação e a gestão de sessões administrativas são providas pelo **Laravel Sanctum**, que oferece um sistema de tokens de *API* leve, porém extremamente seguro para *SPAs*.

A arquitetura de segurança é complementada pelo uso de **Middlewares** customizados, que atuam como interceptadores no pipeline de requisições. Estes componentes verificam não apenas a validade criptográfica do *Bearer Token*, mas também realizam verificações de autorização baseadas em papéis (*Role-Based Access Control - RBAC*), garantindo que apenas usuários com as devidas permissões possam acessar rotas sensíveis de infraestrutura. Conforme postulado por Gamma et al. (1995), o uso de padrões de comportamento como o *Chain of Responsibility* — a base técnica dos *middlewares* — permite processar e validar requisições de forma modular, escalável e segura.

5 IMPLEMENTAÇÃO E ORQUESTRAÇÃO DO ECOSISTEMA OFICINA MASTER

Este capítulo detalha a implementação técnica do Oficina Master, examinando as decisões de engenharia adotadas para resolver os desafios inerentes a um ecossistema *Software as a Service* (SaaS) escalável e multi-tenant. O desenvolvimento foi orientado por dois princípios fundamentais: a separação rigorosa de responsabilidades entre camadas de domínio e infraestrutura, e o uso estratégico de padrões de projeto que garantem a manutenibilidade e a resiliência do sistema frente ao crescimento contínuo de instâncias isoladas. Cada decisão técnica aqui documentada reflete um compromisso com a sustentabilidade arquitetural do produto a longo prazo, priorizando a rastreabilidade, a idempotência dos processos automatizados e a observabilidade centralizada do ecossistema.

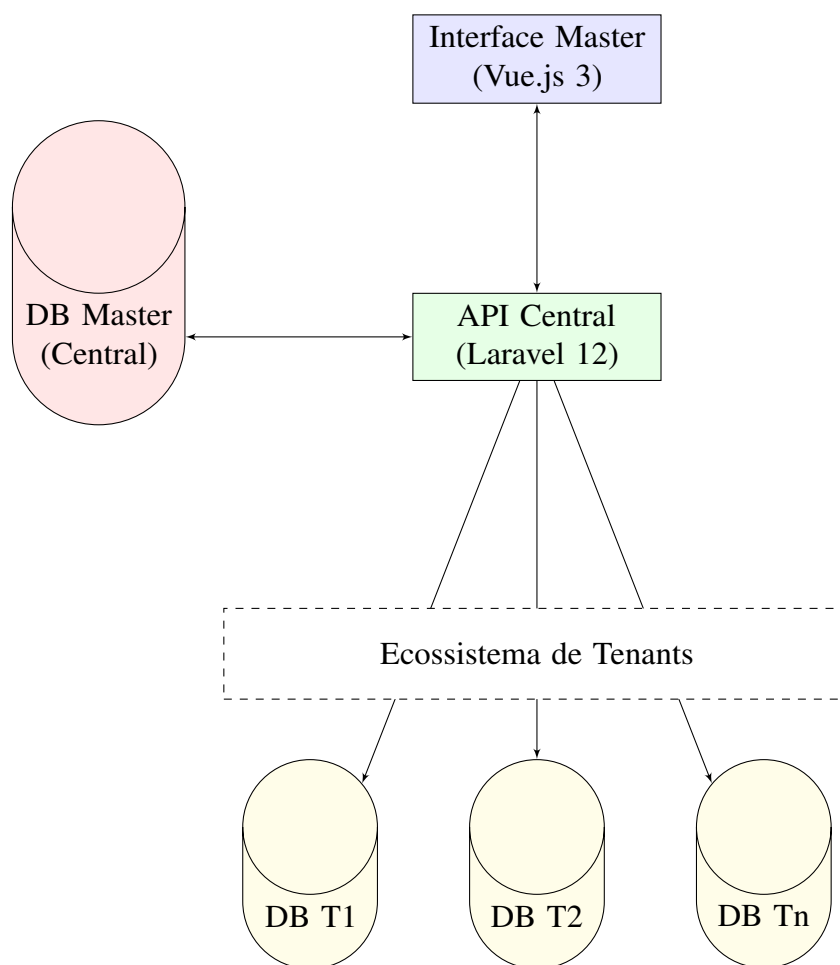
5.1 MACRO ARQUITETURA E ESTRATÉGIA DE ISOLAMENTO DE DADOS

A fundação do Oficina Master é uma arquitetura orientada a serviços que eleva o isolamento de dados ao status de requisito não-funcional primário. A decisão arquitetural central foi a adoção do padrão **Database-per-Tenant**, no qual cada locatário (*tenant*) possui sua própria instância lógica de banco de dados, inteiramente segregada das demais.

A adoção desse padrão, em detrimento de abordagens alternativas como o *Shared Schema* (múltiplos tenants na mesma base, separados por identificador de coluna) ou o *Shared Database, Separate Schema* (bases compartilhadas com schemas isolados), foi uma escolha deliberada com implicações diretas na segurança e na operabilidade. Em um modelo de oficinas mecânicas, os dados de clientes, veículos e transações financeiras possuem natureza sensível e estão sujeitos a regulamentações de privacidade. O isolamento físico elimina classes inteiras de vulnerabilidade, como vazamentos por falhas de predicados em consultas (ausência de cláusula `WHERE tenant_id = ?`) e simplifica drasticamente os processos de backup granular e recuperação de desastres por cliente.

Conforme ilustrado na Figura 3, o Hub Master atua como núcleo de orquestração do ecossistema. A API Central, implementada em *Laravel* 12, mantém as credenciais e os metadados necessários para gerenciar cada instância de banco de dados individual. O banco de dados Master, por sua vez, é exclusivamente dedicado ao armazenamento de dados de orquestração: metadados dos tenants, logs de acesso consolidados e definições dos planos de assinatura. Esta separação de responsabilidades garante que falhas isoladas em uma instância de cliente — sejam elas decorrentes de corrupção de dados, sobrecarga de queries ou manutenção planejada — não comprometam a integridade ou a disponibilidade do restante do ecossistema, satisfazendo o princípio de *blast radius* mínimo em arquiteturas de alta disponibilidade.

Figura 3 – Macro Arquitetura do Ecosystema Oficina Master



Fonte: Própria do autor (2026).

5.2 ORGANIZAÇÃO DO PROJETO E PADRÕES DE IMPLEMENTAÇÃO

O projeto foi estruturado seguindo as convenções modernas do *framework Laravel*, estendendo sua funcionalidade nativa para suportar o multi-tenancy dinâmico. A organização do código é reflexo direto da separação entre lógica de domínio e infraestrutura, princípio fundamental da Arquitetura Limpa (*Clean Architecture*) proposta por Martin (2017).

5.2.1 Estrutura de Pastas e Namespaces

A lógica de negócio foi centralizada na pasta `app/Services`, organizada em domínios coesos e de responsabilidade única:

- `App\Services\Tenants`: Contém o `TenantDatabaseService`, responsável por toda a manipulação de infraestrutura de bancos de dados, incluindo criação, inicialização e teardown de instâncias;
- `App\Services\Licencas`: Agrupa as regras de validação, renovação e controle de status de assinaturas, isolando completamente a lógica de negócio de licenciamento da camada

de persistência;

- App\Http\Controllers\Api: Controladores enxutos (*Lean Controllers*) que atuam exclusivamente como orquestradores de requisições, delegando toda a lógica de negócio aos serviços de domínio correspondentes.

Esta organização tem impacto direto na testabilidade do sistema. Ao desacoplar os serviços de domínio dos controladores HTTP, torna-se possível escrever testes unitários para cada serviço de forma isolada, sem a necessidade de instanciar o contexto completo da aplicação web. A injeção de dependências gerenciada pelo contêiner IoC (*Inversion of Control*) do Laravel viabiliza a substituição de implementações concretas por dublês de teste (*mocks* e *stubs*), elevando a confiabilidade da suíte de testes.

5.2.2 Camada de Modelagem: Eloquent e Relacionamentos de Domínio

O banco de dados Master utiliza modelos Eloquent para gerenciar a orquestração do ecossistema. O modelo Tenant é a entidade central deste domínio, armazenando não apenas as configurações de conexão (host, database, username), mas também o estado operacional de cada instância e seu histórico de eventos. A relevância desta entidade transcende o simples mapeamento de uma tabela: ela é o ponto de convergência entre infraestrutura, licenciamento e faturamento.

Relacionamentos Eloquent complexos foram modelados para vincular cada Tenant a uma Empresa (entidade de negócio) e às suas respectivas Licenças. Esta estrutura relacional permite a execução de consultas analíticas agregadas — como a identificação de tenants com licenças próximas ao vencimento ou com crescimento acelerado de armazenamento — diretamente na camada de modelo, sem a necessidade de lógica de agregação dispersa nos controladores. A decisão de normalizar esta relação em três entidades distintas (Tenant, Empresa, Licença), em vez de consolidá-las em uma única tabela, foi motivada pelo princípio da responsabilidade única e pela antecipação de requisitos futuros, como a possibilidade de um mesmo grupo empresarial gerenciar múltiplos tenants.

5.3 PROVISIONAMENTO AUTOMATIZADO E ORQUESTRAÇÃO DE INSTÂNCIAS

O provisionamento de novas empresas parceiras no ecossistema do Oficina Master constitui um dos processos de maior criticidade operacional da plataforma. Manualmente, este fluxo envolveria a execução sequencial e propensa a erros de comandos de banco de dados, configurações de acesso e inicialização de dados — um processo inviável na escala proposta pelo produto. A solução implementada automatiza integralmente esta sequência, seguindo o princípio de **idempotência**: cada etapa do fluxo pode ser reexecutada sem produzir efeitos colaterais indesejados caso parcialmente concluída.

O fluxo de trabalho de provisionamento, representado formalmente no diagrama de atividades da Figura 4, evidencia a natureza transacional do processo. Cada etapa da cadeia é uma operação atômica; a falha em qualquer ponto aciona o mecanismo de compensação (*rollback*) sobre o banco de dados Master, revertendo os metadados registrados e garantindo que nenhuma instância parcialmente inicializada persista no ecossistema.

Figura 4 – Diagrama de Atividades: Provisionamento Automatizado de Nova Instância



Fonte: Própria do autor (2026).

A escolha por um fluxo **síncrono** em detrimento de uma abordagem assíncrona baseada em filas de mensagens (*message queues*) foi uma decisão arquitetural consciente. Em um sistema de gestão B2B onde o operador aguarda ativamente a confirmação do provisionamento, a latência adicional de poucos segundos é aceitável, ao passo que a complexidade introduzida por um broker de mensagens (como RabbitMQ ou Laravel Horizon) seria desproporcional para a escala atual. Esta decisão está documentada como uma dívida técnica planejada, a ser revisada quando o volume de provisionamentos diários justificar a adoção de processamento assíncrono.

5.3.1 TenantDatabaseService: Motor de Provisionamento e Estratégia de Inicialização

O componente `TenantDatabaseService` encapsula a lógica de baixo nível para criação e inicialização de instâncias, funcionando como a camada de abstração entre a lógica de orquestração de alto nível e as operações diretas de banco de dados. A centralização desta responsabilidade em um único serviço, em vez de distribuí-la pelos controladores, foi motivada pela necessidade de garantir que qualquer variação futura no processo de provisionamento — como a migração para um serviço gerenciado de banco de dados em nuvem — exija modificações em um único ponto do sistema.

O método central, `createAndInitialize`, realiza quatro operações em sequência controlada. Primeiramente, executa o comando `CREATE DATABASE` no servidor de banco de dados

alvo, criando a instância lógica. Em seguida, reconfigura em tempo de execução as conexões do *framework* via `Config::set()`, evitando a necessidade de reinicialização da aplicação ou de arquivos de configuração pré-existentes para cada tenant. A terceira etapa consiste na injeção do arquivo `oficina.dump` através do método `DB::connection()->unprepared()`, garantindo que toda nova empresa inicie suas operações com uma estrutura de dados idêntica, validada e livre de inconsistências de migração incremental. Por fim, as credenciais do administrador inicial são injetadas diretamente na nova base de dados, tornando a instância imediatamente operacional sem intervenção manual adicional.

A opção pelo uso de um *dump* SQL pré-validado, em vez da execução sequencial das *migrations* do Laravel, merece justificativa técnica. Embora as *migrations* representem a abordagem convencional, sua execução sequencial sobre bases recém-criadas introduz risco de inconsistência caso alguma migração intermediária apresente falha. O *dump* garante um estado inicial determinístico e testado, reduzindo a superfície de falha do provisionamento.

5.3.2 Mecanismo de Rollback e Garantias de Consistência

A confiabilidade do processo de provisionamento depende criticamente do comportamento da camada de rollback. Em caso de falha em qualquer das etapas descritas, a transação aberta no banco de dados Master é revertida integralmente, eliminando os registros de metadados parcialmente criados. Contudo, a criação física do banco de dados no servidor MySQL é uma operação DDL (*Data Definition Language*) que, por definição, não é transacional na maioria dos motores de banco de dados relacionais. Isso implica que, em cenários de falha após a criação física mas antes da conclusão do registro de metadados, o sistema prevê uma rotina de coleta de lixo (*garbage collection*) que identifica e remove instâncias físicas órfãs — aquelas presentes no servidor de banco de dados mas sem correspondência na tabela de tenants do Master.

5.4 GESTÃO CENTRALIZADA E HUB DE BANCOS DE DADOS

O monitoramento contínuo de centenas de instâncias de banco de dados impõe requisitos operacionais que vão muito além das ferramentas convencionais de administração de banco de dados. Ferramentas como phpMyAdmin ou clientes SQL genéricos operam sobre uma conexão fixa, exigindo reconexão manual a cada instância inspecionada. O Oficina Master resolve este problema por meio de um Hub centralizado com alternância dinâmica de contexto, desenvolvido como parte integrante da plataforma.

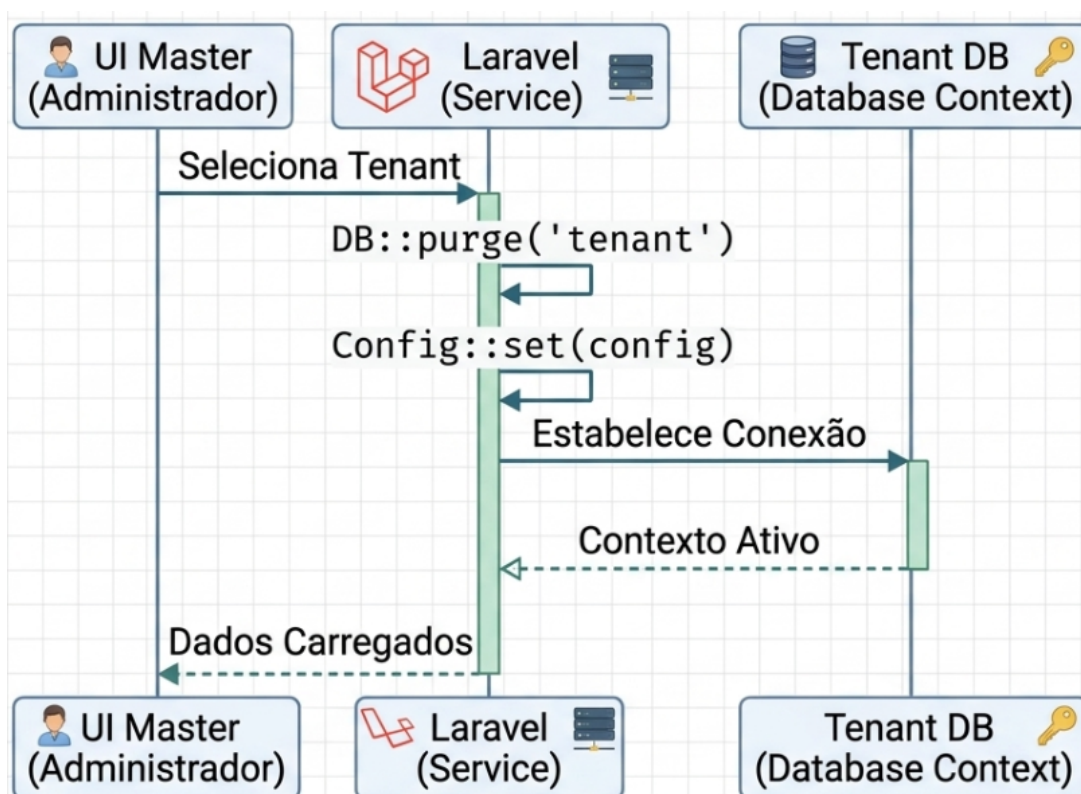
5.4.1 Tenant Switching: Alternância Dinâmica de Contexto de Conexão

O desafio de gerenciar múltiplas conexões de banco de dados em uma única sessão de aplicação foi resolvido através do padrão **Tenant Switching**. A solução abandona o paradigma de conexões fixas configuradas estaticamente no arquivo `.env`, adotando em seu lugar uma estratégia de configuração dinâmica em tempo de execução. O princípio é simples em sua

intenção, mas sofisticado em sua implementação: antes de qualquer operação sobre um tenant específico, o sistema invalida o cache de conexões existente (`DB::purge('tenant')`) e renegocia os parâmetros da conexão nomeada (`tenant`) com as credenciais do banco de dados alvo.

O diagrama de sequência da Figura 5 representa formalmente os atores e as mensagens trocadas neste processo, evidenciando que a operação de alternância é inteiramente transparente para o administrador: a interface Vue.js abstrai a complexidade da troca de contexto, apresentando ao usuário apenas os dados do tenant selecionado.

Figura 5 – Diagrama de Sequência: Alternância Dinâmica de Contexto de Tenant (*Tenant Switcher*)



Fonte: Própria do autor (2026).

Uma implicação arquitetural importante deste padrão é a sua natureza **not thread-safe** em ambientes de servidor com workers concorrentes. Em servidores que utilizam PHP-FPM com múltiplos workers (processos), cada requisição é tratada de forma isolada em seu próprio processo, o que elimina o risco de vazamento de contexto entre tenants. Esta garantia de isolamento por processo é, portanto, uma dependência implícita do modelo de execução do PHP, e deve ser considerada no planejamento de migração para runtimes alternativos, como RoadRunner ou Swoole, que operam com workers persistentes em memória.

5.4.2 Telemetria e Observabilidade das Instâncias

A alternância dinâmica de contexto habilita uma capacidade que vai muito além da simples execução de queries manuais: permite a coleta sistemática de métricas de saúde de cada

instância, alimentando o painel de observabilidade centralizado da plataforma. Através da interface Master, o `DatabaseController` consolida dados de telemetria por instância: quantidade de tabelas e registros ativos, espaço em disco ocupado — discriminado entre dados (*data*) e índices (*index*) — e logs de erro com timestamps de última ocorrência.

Esta visibilidade centralizada tem impacto direto na postura operacional da equipe técnica: em vez de uma abordagem reativa, onde problemas são reportados por clientes afetados, o sistema habilita uma cultura de monitoramento proativo. A identificação de crescimento anômalo de tamanho de banco de dados, por exemplo, pode sinalizar um problema de dados duplicados ou ausência de rotinas de arquivamento, permitindo intervenção antes que o cliente perceba degradação de performance.

5.5 ADMINISTRAÇÃO DE PLANOS E LICENCIAMENTO FLEXÍVEL

A sustentabilidade financeira do ecossistema Oficina Master fundamenta-se em um modelo de planos altamente configurável, cujo design foi orientado pela necessidade de suportar diferentes perfis de clientes — desde oficinas independentes de pequeno porte até redes de franquias com múltiplas filiais. A flexibilidade do modelo de licenciamento é, portanto, não apenas uma decisão comercial, mas uma exigência arquitetural que impacta diretamente a estrutura de dados e a lógica de controle de acesso da plataforma.

5.5.1 Estrutura de Planos e Configuração de Módulos

Cada plano é representado pela entidade `Plano`, que armazena em formato JSON a lista de módulos habilitados para um determinado nível de assinatura. A adoção do tipo JSON para este atributo, em vez de uma tabela relacional de associação entre planos e módulos, foi motivada pela natureza altamente variável da composição de módulos: a criação de uma nova combinação de funcionalidades não exige alterações no schema do banco de dados, reduzindo o custo de manutenção e a possibilidade de migrações disruptivas em produção. Os módulos disponíveis incluem domínios funcionais como Vendas, Estoque, Financeiro e emissão de Nota Fiscal Eletrônica (NF-e), cada qual com impacto direto nas permissões de acesso à API do tenant correspondente.

O `PlanoController` gerencia o ciclo de vida dos modelos de plano, expondo endpoints para a definição de limites operacionais (número máximo de usuários simultâneos e cota de armazenamento), controle de precificação e períodos de renovação, e configuração granular de permissões por módulo. A separação entre a entidade `Plano` e a entidade `Licenca` é uma distinção semântica fundamental: o plano define *o que* está disponível; a licença define *para quem e por quanto tempo*.

5.5.2 Ciclo de Vida do Licenciamento e Controle de Acesso

A licença ativa de um tenant é o artefato que materializa o contrato comercial entre a FK Oficina e cada empresa parceira. Seu ciclo de vida compreende quatro estados: *pendente* (emitida, aguardando ativação), *ativa* (dentro do período de vigência), *expirada* (prazo encerrado, sem renovação) e *suspensa* (bloqueada manualmente por intervenção administrativa). A transição entre esses estados é gerenciada pelo `LicencaService` e verificada em cada requisição recebida pela API do tenant.

O mecanismo de verificação é implementado como um *middleware* do Laravel, posicionado antes das camadas de autenticação e autorização do tenant. Esta decisão de design garante que o controle de licenciamento seja transversal e não possa ser contornado por configurações específicas de rotas ou controladores. Quando uma licença é identificada como expirada ou suspensa, a API do tenant retorna um código HTTP 402 `Payment Required` ou 403 `Forbidden`, respectivamente, acompanhado de uma mensagem estruturada que a interface do usuário final pode interpretar para exibir um aviso contextual ao operador da oficina.

5.6 DASHBOARD ADMINISTRATIVO E INTERFACE REATIVA

O Dashboard Master representa a síntese operacional de toda a arquitetura descrita nas seções anteriores: é a camada de apresentação que transforma dados distribuídos em múltiplas instâncias e domínios em inteligência acionável para o gestor do ecossistema. Construído com `Vue.js 3` e consumindo a API Central do Laravel, o Dashboard atua como o ponto único de verdade (*single source of truth*) para a tomada de decisões operacionais e estratégicas.

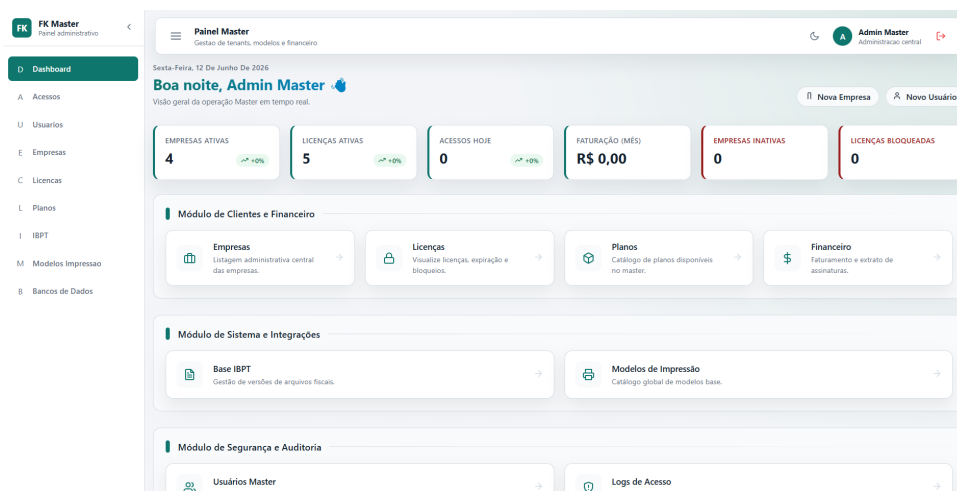
5.6.1 KPIs e Métricas Agregadas por Domínio

O `DashboardController` realiza a agregação de dados provenientes de múltiplos domínios do sistema, calculando indicadores-chave de desempenho (KPIs) que refletem a saúde multidimensional do ecossistema. A organização dos indicadores em quatro domínios distintos — saúde da base de clientes, status de licenciamento, engajamento de uso e saúde financeira — não é apenas uma decisão de UX, mas reflete a estrutura de responsabilidades da equipe que opera a plataforma: cada domínio de KPI corresponde a um fluxo de trabalho diferente, seja ele de suporte técnico, sucesso do cliente, produto ou faturamento.

O indicador de **saúde da base de clientes** consolida o quantitativo de empresas ativas versus inativas, permitindo identificar tendências de churn. O **status de licenciamento** apresenta o total de licenças vigentes e emite alertas sobre vencimentos iminentes, habilitando ações proativas de renovação. O monitoramento de **engajamento e uso**, alimentado pelos logs de acesso (`AuthAccessLog`), fornece dados de atividade diária por tenant, possibilitando a identificação de instâncias subutilizadas — um sinal precoce de risco de cancelamento. Por fim, a dimensão

de **saúde financeira** consolida recebíveis e faturamento mensal através do modelo Pagamento, oferecendo uma visão integrada da performance comercial do ecossistema.

Figura 6 – *Dashboard* Administrativo: Painei Integrado de KPIs e Telemetria



Fonte: Própria do autor (2026).

A interface ilustrada na Figura 6 foi concebida para minimizar o tempo de detecção de anomalias (*Mean Time To Detect* – MTDD), organizando os indicadores de maior criticidade operacional em posição de destaque visual. Esta decisão de design reflete um princípio de UX para ferramentas de monitoramento: a hierarquia visual deve corresponder à hierarquia de impacto no negócio.

5.6.2 Arquitetura Reativa da Interface e Estratégia de Atualização

A escolha do *Vue.js 3* como *framework* de interface não foi arbitrária. O sistema de reatividade baseado em *Proxy* da *Composition API* do Vue 3, em contraste com o sistema de observadores (*watchers*) do Vue 2, oferece granularidade de reatividade por propriedade, reduzindo re-renderizações desnecessárias em componentes que exibem grandes volumes de dados de telemetria. A interface é composta por componentes desacoplados, cada um responsável por um domínio de KPI específico, e se comunica com a API Central via cliente HTTP baseado em *Axios*, com estratégia de polling configurável para atualização periódica dos indicadores sem necessidade de recarregamento de página.

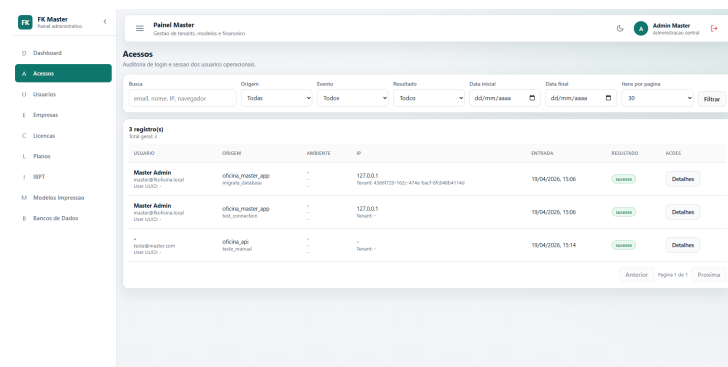
5.7 DEMONSTRAÇÃO DAS INTERFACES DE GESTÃO E EVIDÊNCIAS

A implementação das interfaces de gestão do Hub Master constitui a materialização dos requisitos funcionais e não-funcionais descritos nas seções anteriores. Cada tela foi projetada como uma camada de abstração sobre operações técnicas complexas, tornando acessíveis ao operador ações que, em sua essência, envolvem manipulação de infraestrutura de banco de dados e controle de acesso multi-tenant.

5.7.1 Acesso e Gestão de Usuários

O acesso à plataforma é protegido por uma camada de autenticação robusta (Figura 7), integrada ao *Laravel Sanctum*. O Sanctum fornece autenticação baseada em tokens API de forma simplificada, adequada para aplicações SPA como a desenvolvida. A gestão de usuários é dividida entre usuários administrativos do sistema (Subfigure 8a) e usuários específicos de cada oficina parceira (Subfigure 8b).

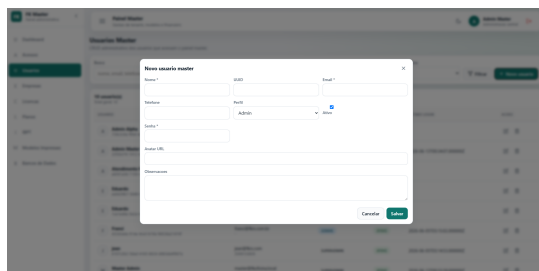
Figura 7 – Interface de Autenticação do Hub Master



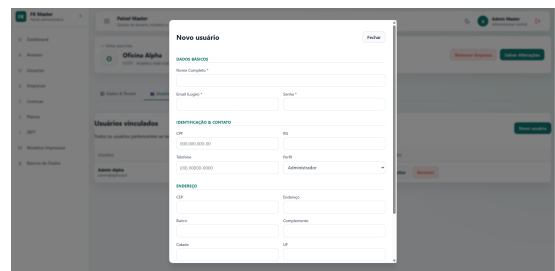
The screenshot shows the 'Acessos' (Accesses) section of the Hub Master interface. It displays a table of login attempts with columns for 'USUARIO', 'ORIGEM', 'AMBIENTE', 'IP', 'ENTRADA', 'RESULTADO', and 'AÇÕES'. The table contains three entries, all with a 'Sucesso' (Success) status.

USUARIO	ORIGEM	AMBIENTE	IP	ENTRADA	RESULTADO	AÇÕES
Master Admin	oficina_master_app	-	127.0.0.1	18/04/2025, 15:08	Sucesso	Detalhes
Master Admin	oficina_master_app	-	127.0.0.1	18/04/2025, 15:08	Sucesso	Detalhes
Master Admin	oficina_app	-	127.0.0.1	18/04/2025, 15:14	Sucesso	Detalhes

Fonte: Própria do autor (2026).



(a) Criação de Usuário Master



(b) Criação de Usuário de Empresa

Figura 8 – Interfaces de Gestão de Usuários e Controle de Acesso

Fonte: Própria do autor (2026).

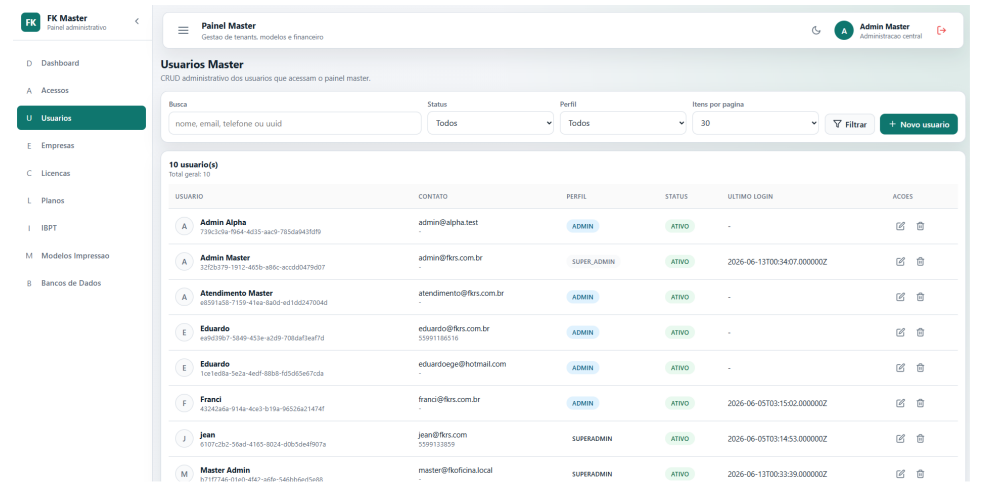
5.7.2 Gestão de Tenants, Planos e Licenciamento

A gestão individual de cada cliente é viabilizada pela interface de listagem de *tenants* (Figura 10). O administrador obtém, em uma única visão, o status operacional de cada banco de dados (Figura 11) e a validade das licenças correspondentes. A flexibilidade comercial é garantida pelas interfaces de criação de planos (Subfigure 13a) e emissão de licenças (Subfigure 13b), que permitem configurar dinamicamente os recursos disponíveis para cada oficina.

5.7.3 Evidências de Desenvolvimento e Controle de Versão

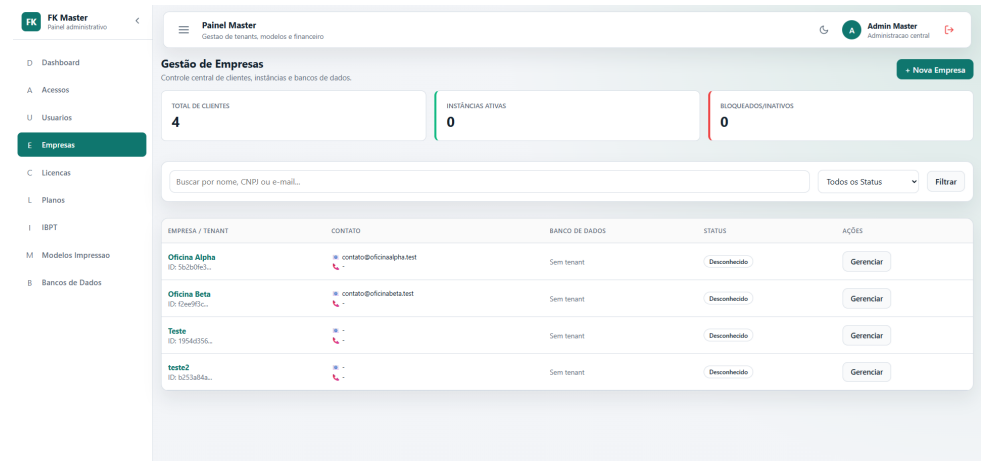
O código-fonte integral encontra-se disponível no repositório registrado no Apêndice ???. A análise deste registro permite verificar a autoria e a consistência das contribuições, constituindo

Figura 9 – Listagem Geral de Usuários Cadastrados no Sistema



Fonte: Própria do autor (2026).

Figura 10 – Listagem Geral de Empresas Cadastradas

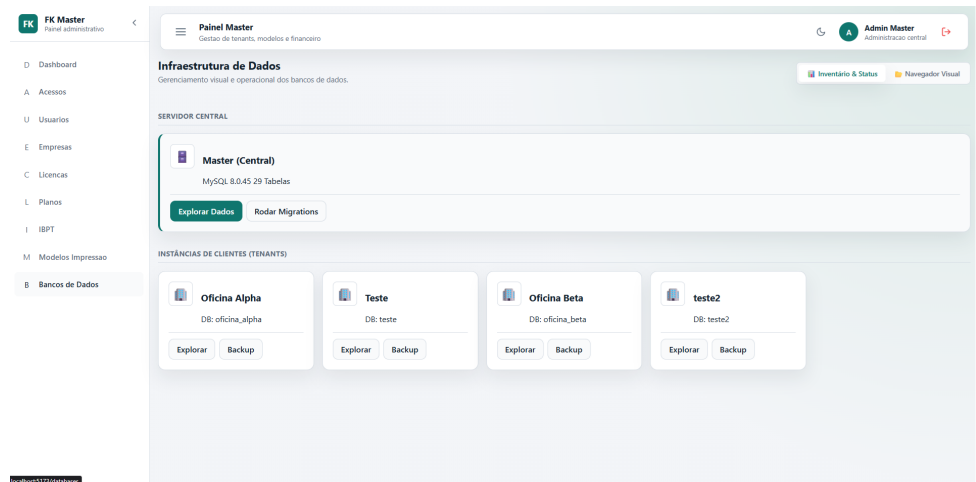


Fonte: Própria do autor (2026).

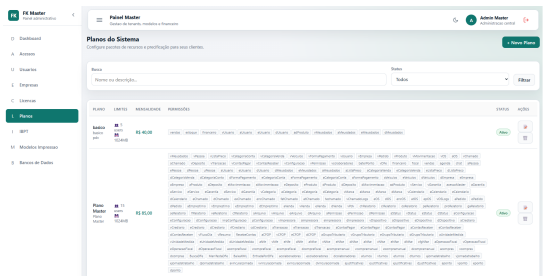
um registro auditável do processo de engenharia aplicado. A transição entre as fases de pesquisa, prototipação e implementação final é visível no histórico de commits, validando a metodologia descrita anteriormente.

Esta camada de gestão visual e documental é o que eleva o Oficina Master da categoria de ferramenta técnica de automação à de plataforma estratégica de gestão de ecossistemas SaaS: ao abstrair a complexidade dos mecanismos técnicos em uma experiência de usuário coerente, o sistema habilita que decisões de infraestrutura e negócio sejam tomadas com base em evidências em tempo real.

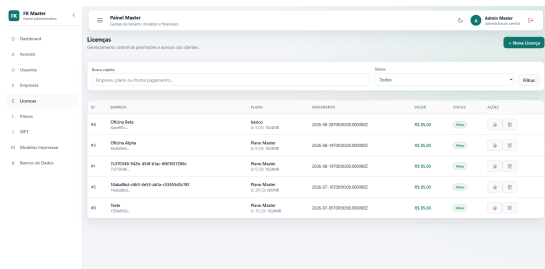
Figura 11 – Monitoramento de Instâncias de Bancos de Dados



Fonte: Própria do autor (2026).



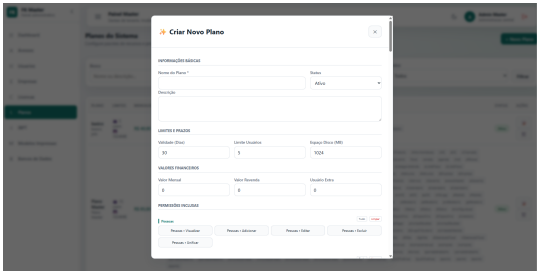
(a) Listagem de Planos



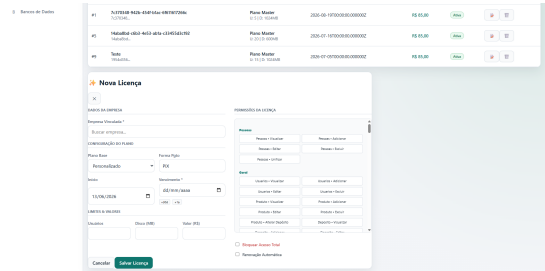
(b) Listagem de Licenças

Figura 12 – Visualização Consolidada de Planos e Licenças no Ecossistema

Fonte: Própria do autor (2026).



(a) Criação de Planos

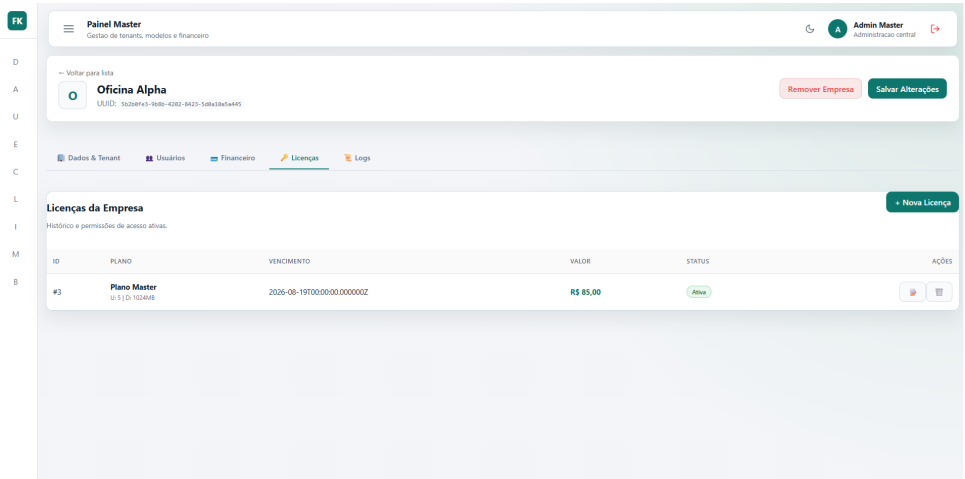


(b) Emissão de Licença

Figura 13 – Interfaces de Configuração Comercial e Emissão de Licenças

Fonte: Própria do autor (2026).

Figura 14 – Interface de Alteração e Ajuste de Plano de Empresa



Fonte: Própria do autor (2026).

6 CONSIDERAÇÕES FINAIS

A conclusão deste estágio supervisionado na FK Oficina marca o encerramento de um ciclo de intenso aprendizado técnico e amadurecimento profissional. O desenvolvimento do projeto Oficina Master permitiu a transposição de conceitos teóricos fundamentais da Ciência da Computação para uma solução de engenharia de software de alta complexidade, operando em um cenário real de *SaaS* multi-tenant.

6.1 ANÁLISE CRÍTICA DAS DECISÕES ARQUITETURAIS

A escolha pelo modelo de isolamento de banco de dados (*database-per-tenant*) provou-se acertada, oferecendo o equilíbrio ideal entre segurança da informação e flexibilidade operacional. Embora essa decisão tenha imposto desafios significativos na orquestração de conexões dinâmicas, o desenvolvimento do *TenantDatabaseService* e do mecanismo de *Tenant Switching* mitigou com sucesso essa complexidade. A automação desses processos eliminou intervenções manuais propensas a erros, permitindo que a infraestrutura escale de forma linear com o crescimento da base de clientes.

A utilização da *Service Layer* em conjunto com *Lean Controllers* no *backend* Laravel 12 garantiu uma base de código modular e de fácil manutenção. No *frontend*, a adoção do Vue.js 3 com *Composition API* e *Pinia* proporcionou uma interface reativa e performática, essencial para uma ferramenta de gestão administrativa que demanda alta densidade de dados e respostas instantâneas.

6.2 AVALIAÇÃO DO IMPACTO NO ECOSISTEMA FK OFICINA

O Oficina Master deixou de ser um projeto de desenvolvimento para tornar-se o núcleo operacional da FK Oficina. Os resultados tangíveis incluem:

- **Agilidade no Provisionamento:** A redução do tempo de configuração de novos clientes de horas para poucos minutos, através da automação total da criação e inicialização de bancos de dados;
- **Visibilidade Estratégica:** A centralização de indicadores de saúde técnica e status de licenciamento em um *Dashboard* consolidado, permitindo uma gestão proativa e baseada em dados reais do ecossistema;
- **Flexibilidade Comercial:** A capacidade de gerenciar planos e módulos de forma granular, facilitando a criação de novas ofertas de mercado sem necessidade de alterações no código-fonte.

6.3 REFLEXÃO SOBRE O CRESCIMENTO PROFISSIONAL

A imersão no cotidiano de uma *software house* focada em produtos de missão crítica foi fundamental para consolidar competências em injeção de dependências, padrões de projeto, segurança em APIs REST e gestão de infraestrutura em nuvem. O desafio de garantir a integridade de dados em um ambiente distribuído e a responsabilidade de desenvolver ferramentas que sustentam a operação de outras empresas foram os catalisadores de um salto qualitativo na minha formação como Cientista da Computação.

Para evoluções futuras, vislumbra-se a implementação de camadas de telemetria preditiva e a migração de rotinas pesadas para arquiteturas *serverless*, buscando otimização contínua de custos e performance. Em suma, o estágio cumpriu integralmente seu papel de integrar academia e mercado, resultando em uma solução robusta que agrega valor tecnológico e estratégico para a organização.

REFERÊNCIAS

- BEZEMER, C. P.; ZAIDMAN, A. Multi-tenant software architectures: A systematic literature review. *Journal of Software: Evolution and Process*, 2010. Disponível em: <<https://doi.org/10.1002/smr.445>>. Citado na página 18.
- FOWLER, M. *Patterns of Enterprise Application Architecture*. [S.l.]: Addison-Wesley Professional, 2002. Citado na página 19.
- GAMMA, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. [S.l.]: Addison-Wesley, 1995. Citado na página 20.
- KREBS, R.; MOMM, C.; KOUNEV, S. Architectural concerns in multi-tenant saas applications. *Proceedings of the 2nd International Conference on Cloud Computing and Services Science*, 2012. Citado na página 18.
- MARTIN, R. C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. [S.l.]: Prentice Hall, 2017. Citado 2 vezes nas páginas 19 e 22.
- OTWELL, T. *Laravel Documentation: The PHP Framework for Web Artisans*. [s.n.], 2024. Acesso em: 02 jun. 2026. Disponível em: <<https://laravel.com/docs>>. Citado na página 18.
- PRESSMAN, R. S. *Engenharia de Software: uma abordagem profissional*. 7. ed. Porto Alegre: McGraw-Hill, 2011. Citado 2 vezes nas páginas 15 e 18.
- YOU, E. *Vue.js 3 Documentation: The Progressive JavaScript Framework*. [s.n.], 2024. Acesso em: 02 jun. 2026. Disponível em: <<https://vuejs.org/>>. Citado na página 19.

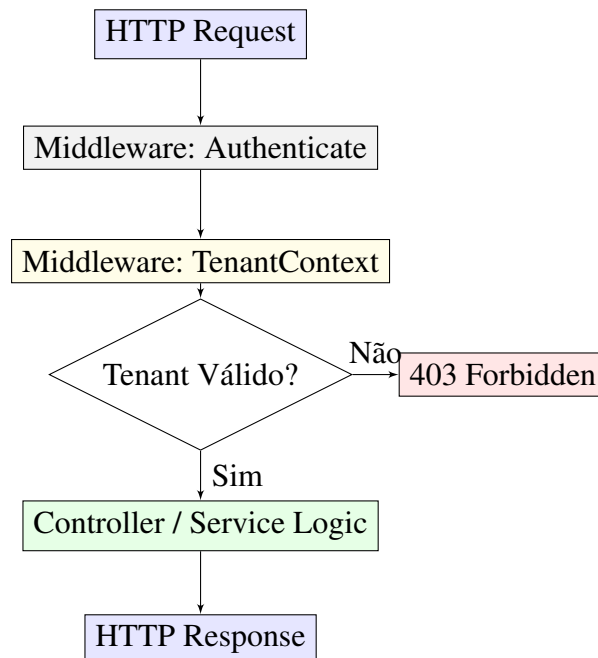
APÊNDICE A – DIAGRAMAS E FLUXOS TÉCNICOS AVANÇADOS

Este apêndice aprofunda-se na arquitetura interna e nos fluxos de trabalho que sustentam a operação do Oficina Master.

A.1 PIPELINE DE REQUISIÇÃO E MIDDLEWARE DE CONTEXTO

A Figura 15 ilustra como cada requisição é processada e como o contexto do banco de dados é injetado dinamicamente antes de atingir a lógica de negócio.

Figura 15 – Pipeline de Processamento de Requisição com Injeção de Contexto



Fonte: Própria do autor (2026).

A.2 ARQUITETURA DE ESTADO NO FRONTEND (PINIA STORES)

O *frontend* utiliza uma arquitetura descentralizada de estados. A ?? descreve a hierarquia das principais *stores* utilizadas.

Fonte: Própria do autor (2026).

A.3 ESTRUTURA DE DIRETÓRIOS DO PROJETO (BACKEND)

A organização das pastas segue o padrão de arquitetura em camadas implementado no estágio:

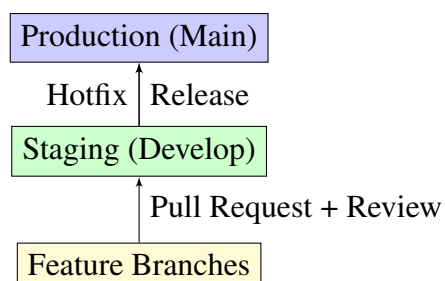
- `app/Http/Controllers/Api`: Controladores REST;

- `app/Models`: Modelos Eloquent do banco Master;
- `app/Services`: Camada de negócio (Onde reside o `TenantDatabaseService`);
- `database/DUMPS`: Esquema SQL base para novos tenants;
- `routes/api.php`: Definição de todos os endpoints administrativos.

A.4 FLUXO DE TRABALHO GITFLOW

A Figura 17 resume a estratégia de ramificação utilizada para garantir a estabilidade do sistema Oficina Master em produção.

Figura 17 – Estratégia de Ramificação e Ciclo de Vida do Código



Fonte: Própria do autor (2026).

ANEXO A – DOCUMENTAÇÃO DE REFERÊNCIAS TÉCNICAS

Este anexo apresenta detalhes técnicos exaustivos sobre as interfaces de integração, padrões de resposta e estruturas de dados do ecossistema Oficina Master.

A.1 DICIONÁRIO DE DADOS DO HUB MASTER

Abaixo detalham-se as propriedades das entidades centrais de orquestração.

A.1.1 Entidade: Tenant (Configurações de Instância)

Campo	Tipo	Descrição
uuid	char(36)	Identificador único global (Primary Key).
name	varchar(255)	Nome fantasia do tenant no sistema.
database	varchar(60)	Nome lógico do esquema no servidor MySQL.
url_banco	varchar(255)	Host/IP do servidor de banco de dados.
username	varchar(60)	Usuário com permissões de owner no banco.
password	text	Senha criptografada (AES-256).
status	enum	Estado operacional (ativo, bloqueado, desativado).
licenca_valida_ate	date	Cache da data de expiração da licença atual.

A.2 ESPECIFICAÇÃO COMPLETA DE ENDPOINTS (API MASTER)

A comunicação entre o *frontend* e o *backend* é realizada via HTTPS utilizando o padrão JSON.

A.2.1 Autenticação e Segurança

- POST /api/auth/login: Autenticação de usuários Master (Retorna Bearer Token);
- GET /api/auth/me: Retorna dados do perfil logado e permissões;
- POST /api/auth/logout: Invalida o token de acesso atual.

A.2.2 Gestão de Tenants e Infraestrutura

- GET /api/empresas: Listagem paginada com filtros por status e nome;
- POST /api/empresas/migrate/{id}: Dispara o *job* de inicialização forçada de banco;

- GET /api/database/tables/{uuid}: Lista tabelas e contagem de registros de um banco isolado;
- POST /api/database/test-connection: Valida credenciais antes da persistência.

A.3 PADRÃO DE RESPOSTA JSON (STANDARD API RESPONSE)

Todas as respostas da API seguem o esquema estruturado abaixo para facilitar o consumo pelo *frontend*:

```
1 {  
2   "status": true,  
3   "code": "SUCCESS",  
4   "data": { ... },  
5   "message": "Operação realizada com sucesso",  
6   "meta": {  
7     "current_page": 1,  
8     "total": 150  
9   }  
10 }
```

Listing A.1 – Exemplo de Resposta de Sucesso

A.4 CONSULTAS DE TELEMETRIA SQL (REFERÊNCIA)

Para a monitoria dos bancos de dados, o sistema utiliza consultas de inspeção de esquema:

```
1 SELECT  
2   table_name AS "Tabela",  
3   table_rows AS "Registros",  
4   round(((data_length + index_length) / 1024 / 1024), 2) AS "  
5     Tamanho (MB)"  
6 FROM information_schema.TABLES  
7 WHERE table_schema = 'db_cliente_uuid'  
8 ORDER BY (data_length + index_length) DESC;
```

Listing A.2 – Query de Inspeção de Tamanho de Tabelas